

MiniCamp Puzzle

Comprehensive Design Document

Version 1.0

May 2026

C# Windows Forms (.NET Framework 4.7.2)

Desktop Application

Table of Contents

1.	Introduction	3
2.	System Architecture	4
3.	Component Design	6
4.	Database Design	9
5.	UI/UX Design	10
6.	Theme System Design	12
7.	Code Execution Engine	13
8.	Data Flow & Navigation	15
9.	Security Considerations	16
10.	Deployment	17
11.	Future Enhancements	18

1. Introduction

1.1 Project Overview

MiniCamp Puzzle is a desktop-based interactive coding puzzle platform built using C# and Windows Forms (WinForms) on the .NET Framework 4.7.2. The application allows users to register, log in, browse programming problems categorized by difficulty level (Easy, Medium, Hard), write and submit C# code solutions in an embedded editor, and receive real-time pass/fail evaluation against predefined test cases. The platform features a modern dual-theme system (Dark/Light), custom-drawn visual elements, and an event-driven navigation architecture.

1.2 Goals & Objectives

The primary objectives of the MiniCamp Puzzle application are:

- Provide an accessible platform for practising programming problem-solving skills offline.
- Offer instant feedback on submitted solutions through automated test case evaluation.
- Support progressive learning through tiered difficulty levels (Easy/Medium/Hard).
- Deliver a polished user experience with a modern dark/light theme and smooth navigation.
- Demonstrate practical use of the Roslyn compiler API for in-memory code compilation and execution.

1.3 Technology Stack

Language:	C# 7.3+
Framework:	.NET Framework 4.7.2
UI Technology:	Windows Forms (WinForms)
Code Compilation:	Roslyn (Microsoft.CodeAnalysis.CSharp v5.3.0)
Database:	SQLite 3 (System.Data.SQLite.Core v1.0.119.0)
IDE:	Visual Studio 2022
Target OS:	Windows 7+

1.4 Target Audience

- Students learning programming fundamentals (loops, arrays, strings, math).
- Hobbyists preparing for coding interviews with classic algorithm problems.
- Educators seeking a simple offline platform for classroom coding exercises.

2. System Architecture

2.1 High-Level Architecture

MiniCamp Puzzle follows a monolithic single-process desktop application architecture. The system is organized into a layered structure with clear separation of concerns: Presentation Layer (Forms/UI), Service Layer (business logic, code execution), Data Access Layer (database operations), and the Theme/Infrastructure layer providing cross-cutting concerns.

The architectural layers are described below:

Presentation Layer

Contains all Windows Forms (UserControls) and custom components (Toolbar, PanelStars). Responsible for user interaction, navigation, and visual rendering. Each form encapsulates its own UI logic and communicates upward via C# events.

Service Layer

Contains the CodeRunner orchestrator and ConnactionCmd process wrapper. This layer implements the core business logic for code compilation, test execution, and result comparison.

Data Access Layer

Contains DatabaseManager (connection/initialization) and UserManagement (CRUD operations). Uses raw ADO.NET with SQLite for data persistence.

Code Execution Layer

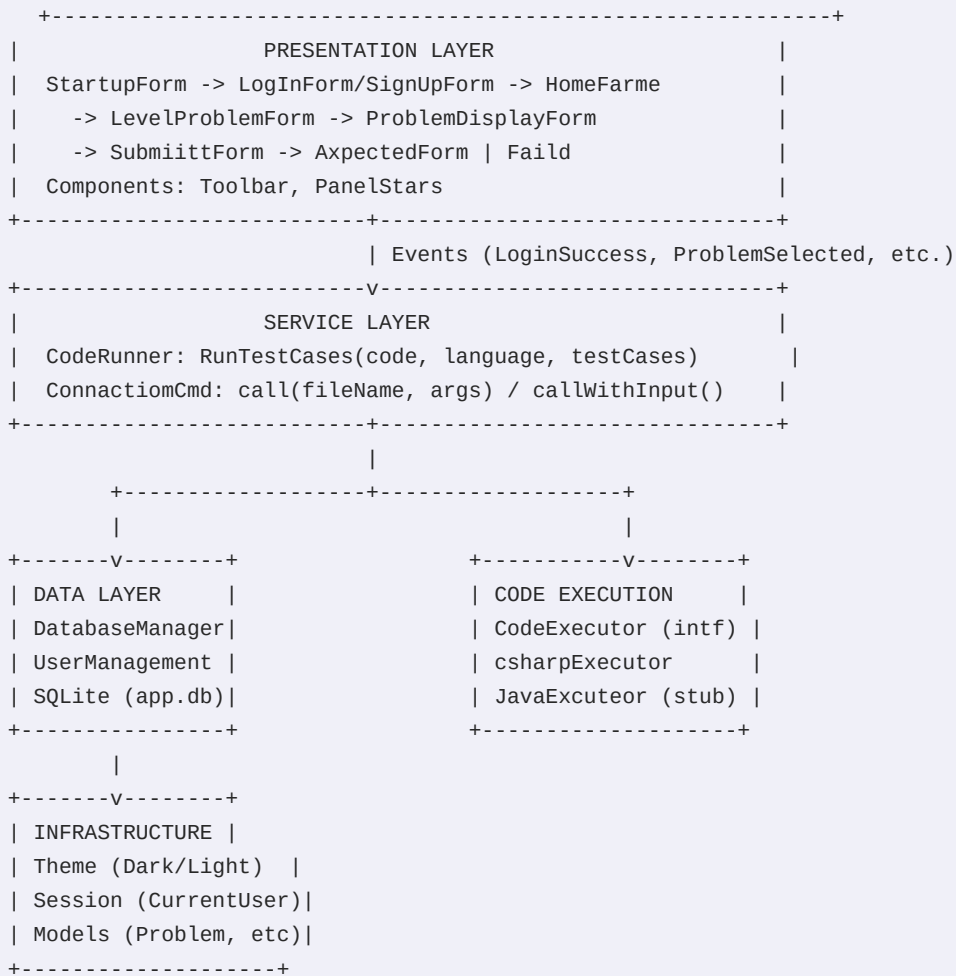
Contains the CodeExecutor interface and its implementations (csharpExecutor for C# via Roslyn, JavaExcuteor as a stub). Provides an abstraction over compilation and execution strategies.

Infrastructure / Cross-Cutting

Contains the Theme system (ITheme, DarkTheme, LightTheme, Theme), Session state (CurrentUser), and data models (Problem, TestCase, Example, etc.).

2.2 Architectural Diagram (Textual)

The following diagram illustrates the layered architecture and data flow:



2.3 Design Patterns

Event-Driven Architecture

Navigation between forms is driven entirely by C# events. Each UserControl publishes events (e.g., LoginSuccess, ProblemSelected, TestResultsReady) that the MainForm subscribes to, enabling loose coupling between UI components.

Strategy Pattern (Code Execution)

The CodeExecutor interface defines the contract for code execution. csharpExecutor and JavaExcuteor are concrete strategies. This allows adding new language support without modifying existing code.

Static Singleton (Session)

CurrentUser provides a static, globally accessible session store for the logged-in user. Properties include Username, Country, Gender, and score.

Template Method (Theme Application)

The Theme.Apply() method recursively traverses the control tree and applies styling based on the active ITheme implementation, following a template-method-like approach.

3. Component Design

3.1 Forms (Presentation Layer)

All forms inherit from `UserControl` and are hosted within a single `MainForm` container. Navigation is managed by pushing/popping `UserControl` instances onto a navigation stack (`backStack`).

MainForm

Root form. Manages navigation stack, hosts all `UserControl` pages, subscribes to page events. Contains the custom `Toolbar`.

StartupForm

Landing/splash screen. Displays application logo, animated title text, and navigation buttons for Login and Sign-Up. Publishes `LoginRequested`, `SignUpRequested`.

LoginForm

Login page with username/password fields, password visibility toggle, and validation. Publishes `LoginSuccess(username)`, `BackRequested`.

SignUpForm

Registration form with username, password, country `ComboBox` (6 countries), gender `RadioButton` group (Male/Female). Publishes `SignUpSuccess`, `BackRequested`.

HomeFarme

Dashboard after login. Contains navigation buttons for Problems, User Profile, and Tutorial. Publishes `UserRequested`, `ProblemsRequested`.

LevelProblemForm

Difficulty selector with Easy/Medium/Hard buttons and a `ComboBox` filtered by selected difficulty. Color-coded items. Publishes `ProblemSelected(string)`.

ProblemDisplayForm

Shows full problem description, input/output specs, constraints, and an example with explanation. Pattern-type problems display `PanelStars` instead of text output. Publishes `SolveRequested`, `HomeRequested`, `UserRequested`, `BackRequested`.

SubmiittForm

Code editor with Consolas-font `TextBox`, language selector (C#/Java), and Run button. Sends code to `CodeRunner` and publishes `TestResultsReady`.

AxpectedForm

Success screen with green checkmark animation and congratulatory message. Publishes `HomeRequested`, `BackRequested`.

Faild

Failure screen showing animated test case result panels (green PASSED / red FAILED). Background image and `GameOver` overlay. Publishes `BackRequested`.

UsreForm

Profile page displaying username, score, gender, country, and gender-specific image. Publishes HomeRequested.

3.2 Custom Components

Toolbar

A custom-drawn Panel that acts as a title bar. Displays the application name, a gear button for theme switching (context menu with Dark Theme / Light Theme options), and a close button (X). Supports window dragging via mouse events. Integrated into MainForm.

PanelStars

A custom Panel override that overrides OnPaint to draw a triangular arrangement of star polygons in olive green using GDI+ drawing primitives. Used exclusively by ProblemDisplayForm to visually represent pattern-based problems (e.g., printing pyramid patterns).

3.3 Data Models (ProblemClasses)

The following data model classes define the problem structure:

Problem

Title, Input, Output, Type, Description, Level, TestCase list, Example, Constraints. The core problem definition model.

TestCase

Input (string), Output (object). Represents a single test case with expected output. ToString() returns formatted "{input} -> {output}".

Example

Input, Output, Explanation. Used for the worked example shown on the problem display page.

ProblemLoadReadJs

Static utility class using JavaScriptSerializer to deserialize ProblemJs.json into a List<Problem>. Method getProblemByName(string title) retrieves a problem by its title.

3.4 Problem Data (ProblemJs.json)

The application ships with 8 predefined coding problems in JSON format, stored in ProblemClasses/ProblemJs.json. The problems cover a range of concepts:

- Program to Print Full Pyramid Pattern (Star Pattern) - Easy - Pattern/loops
- Program to find if a character is vowel or Consonant - Easy - String/conditionals
- Print Fibonacci Series - Easy - Math/loops
- Floor in a Sorted Array - Medium - Array/binary search
- Move All Zeroes to End - Medium - Array/two-pointer
- T-primes - Medium - Number theory/primes
- Happy Number - Hard - Math/cycle detection
- Lucky Numbers in a Matrix - Hard - Matrix/set operations

4. Database Design

4.1 Database Engine

The application uses SQLite 3 as its embedded database engine via the System.Data.SQLite.Core NuGet package (v1.0.119.0). The database file (app.db) is stored in the Database/ directory relative to the application executable.

4.2 Schema Design

The database contains a single table:

```
CREATE TABLE users (  
  id          INTEGER PRIMARY KEY AUTOINCREMENT,  
  username    VARCHAR NOT NULL UNIQUE,  
  password    VARCHAR NOT NULL,  
  Country     VARCHAR NULL,  
  Gender      VARCHAR NULL  
);
```

4.3 Entity Description

id	INTEGER	PRIMARY KEY	Auto-incremented unique identifier for each user
username	VARCHAR	NOT NULL, UNIQUE	User login name, must be unique across all users
password	VARCHAR	NOT NULL	User password (stored as plaintext - see Security section)
Country	VARCHAR	NULLABLE	User country selection (Egypt, Jordan, Lebanon, Palestine, UK, US)
Gender	VARCHAR	NULLABLE	User gender selection (Male, Female)

4.4 Database Manager (DatabaseManager.cs)

DatabaseManager is a static class that manages the SQLite connection lifecycle. On initialization (InitializeDatabase), it creates the database file if it does not exist, creates the users table, attempts ALTER TABLE migrations for Country and Gender columns (to handle legacy schemas), and seeds the default admin/admin user if no users exist in the table.

4.5 User Management (UserManagement.cs)

UserManagement provides static methods for user CRUD operations:

- SignUp(username, password, country, gender) - Creates a new user account.
- SignIn(username, password) - Authenticates user credentials against the database.
- GetUserDetails(username) - Retrieves user profile information.
- UsernameExists(username) - Checks if a username is already taken (used during sign-up validation).

Note: No password hashing is implemented. Passwords are stored as plaintext VARCHAR. This is acceptable for a local educational application but should not be used in production without cryptographic enhancements (see Section 9: Security Considerations).

5. UI/UX Design

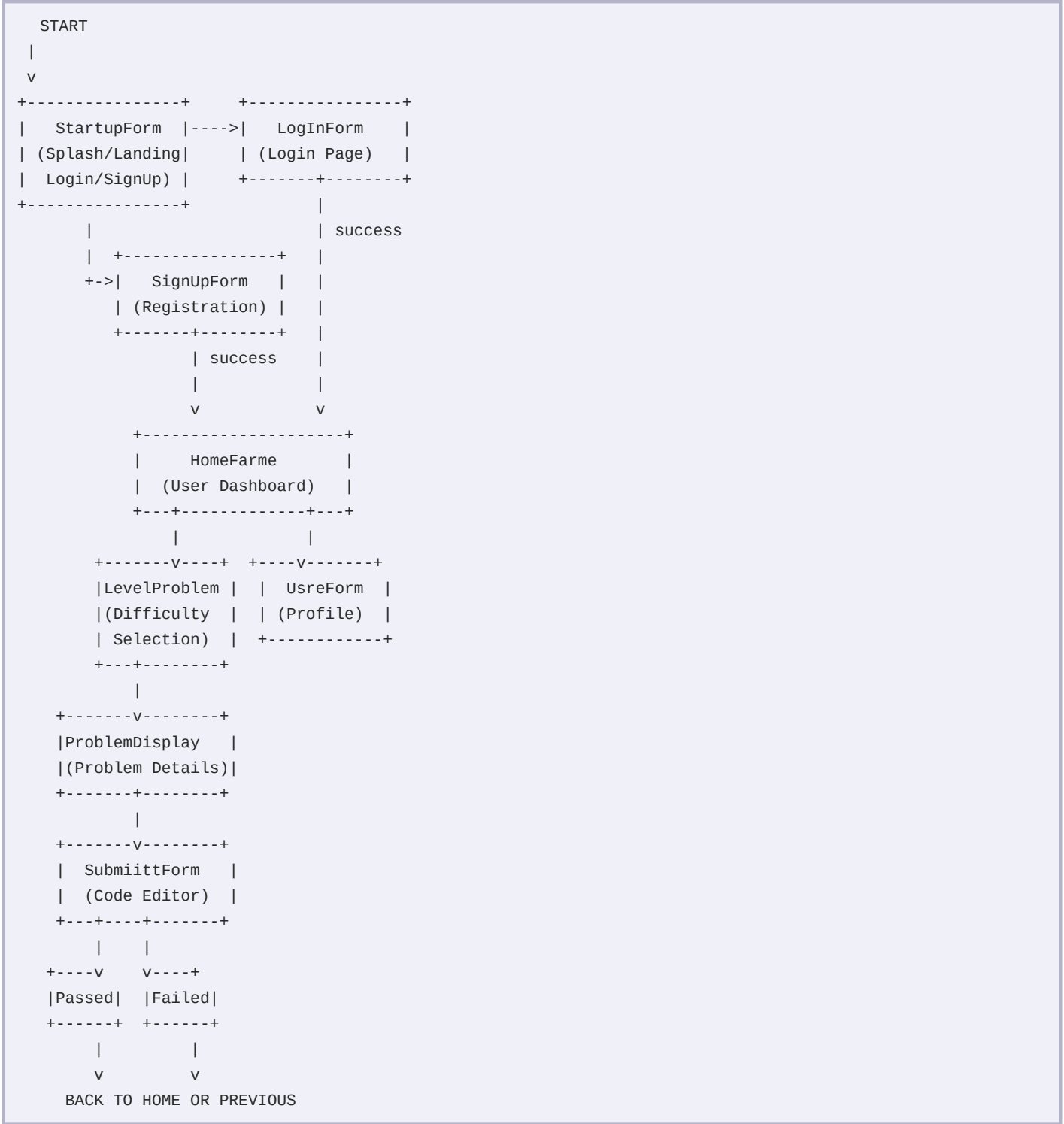
5.1 Visual Identity

MiniCamp Puzzle employs a modern flat-design aesthetic with the following visual characteristics:

- Dual-theme system: Dark theme (dark backgrounds, purple accent, white text) and Light theme (light purple backgrounds, purple accents, dark text).
- Custom title bar (Toolbar component) replacing the standard Windows title bar for a unified appearance.
- Purple (#6C3EB8) as the primary accent color across both themes.
- Consistent border radius simulation through color-matched panels.
- Custom-drawn star patterns for visual representation of pattern problems.

5.2 Navigation Flow

The user navigation flow through the application is as follows:



5.3 Page Descriptions & Layout

StartupForm

Centered layout with application title animation at the top, descriptive subtitle text, and two prominent buttons: Login and Sign Up. Clean background with minimal decoration.

LogInForm

Two TextBox inputs (username, password) with placeholder-style labels. Password field has a visibility toggle button (eye icon). Login button is disabled until both fields are filled. A Back button returns to StartupForm. Error messages

shown for invalid credentials.

SignUpForm

Form-like layout with fields for username, password, country (ComboBox with 6 options: Palestine, Jordan, Lebanon, Egypt, UK, US), and gender (RadioButton: Male/Female). Username availability is validated before submission. Back button for navigation.

HomeFarme

Dashboard-style layout with large icon buttons: Problems (main action), User Profile, and Tutorial. Clean card-based design with clear visual hierarchy.

LevelProblemForm

Three large colored buttons for difficulty selection: Easy (green), Medium (orange), Hard (red). A ComboBox below lists available problems filtered by selected difficulty. Items are color-coded by type. A Select button opens the chosen problem.

ProblemDisplayForm

Structured layout showing: problem title, description text, Input/Output specifications, Constraints, Example with explanation. For pattern-type problems, a PanelStars visual is rendered. Action buttons: Solve, Back, Home, User (profile).

SubmiittForm

Large multi-line TextBox (Consolas font) for code input. Language selector ComboBox (C# / Java). Run button triggers compilation and test execution. Results are processed asynchronously and navigation to success/failure page follows.

AxpectedForm

Celebration screen with green checkmark, congratulatory text, and navigation options to return Home or go Back.

Faild

Test-by-test result display. Each test case is shown as a card with: test case label, expected output, actual output, and a PASSED (green) or FAILED (red) badge. Panels animate in sequentially. Background image and "Game Over" overlay add visual flair.

UsreForm

Profile card showing: username, score, gender-specific image (Male.jpg or Famale.jpg), country, and score display. Home button for navigation.

5.4 Color Palette

Theme	Background	Accent	Text	Button
Dark	#1A1A2E	#6C3EB8	#FFFFFF	#2D2D44
Light	#F5F0FF	#6C3EB8	#1A1A2E	#E8E0F0

6. Theme System Design

6.1 Architecture

The theme system follows a Strategy-like pattern with a clear interface and interchangeable implementations:

ITheme (Interface)

Defines properties for all visual styling attributes: FormBackColor, ControlBackColor, ButtonBackColor, ButtonHoverBackColor, AccentColor, TextColor, SecondaryTextColor, BorderColor, InputBackColor, SelectionForeColor.

DarkTheme : ITheme

Implements ITheme with a dark color palette: near-black backgrounds (#1A1A2E), dark control backgrounds (#2D2D44), purple accent (#6C3EB8), and white text (#FFFFFF).

LightTheme : ITheme

Implements ITheme with a light color palette: light purple backgrounds (#F5F0FF), light control backgrounds (#E8E0F0), purple accent (#6C3EB8), and dark text (#1A1A2E).

Theme (Static Manager)

Static class managing the active theme state. Provides ToggleTheme(control) and SetTheme(control, themeType) methods. The Apply(control) method recursively traverses the control tree, applying styling to all supported control types including: Button, Label, TextBox, ComboBox, DataGridView, GroupBox, Panel, ListBox, CheckBox, RadioButton.

6.2 Theme Switching Flow

1. User clicks gear button on Toolbar
2. Context menu appears with "Dark Theme" / "Light Theme" options
3. User selects a theme
4. Theme.SetTheme(this, selectedTheme) is called on MainForm
5. Theme.Apply(control) recursively processes all child controls
6. Each control's BackColor, ForeColor, FlatStyle, etc. are updated
7. UI instantly reflects the new color scheme

6.3 Supported Control Types

The Theme.Apply() method handles the following control types, each with specific styling rules:

- Button - BackColor, ForeColor, FlatStyle, FlatAppearance.MouseOverBackColor, FlatAppearance.BorderColor
- Label - BackColor, ForeColor
- TextBox - BackColor, ForeColor, BorderStyle
- ComboBox - BackColor, ForeColor, FlatStyle
- DataGridView - BackgroundColor, ForeColor, GridColor, ColumnHeadersDefaultCellStyle, EnableHeadersVisualStyles
- GroupBox - BackColor, ForeColor

- Panel - BackColor
- ListBox - BackColor, ForeColor, BorderStyle
- CheckBox - BackColor, ForeColor, FlatStyle
- RadioButton - BackColor, ForeColor, FlatStyle

7. Code Execution Engine

7.1 Architecture Overview

The code execution subsystem enables users to write C# code in the embedded editor, compile it in-memory using the Roslyn compiler API, execute it against predefined test cases, and compare outputs. The architecture abstracts language-specific execution behind the `CodeExecutor` interface, allowing future extensibility (e.g., Java, Python).

7.2 Components

CodeExecutor (Interface)

Defines the contract: `string ExecuteCode(string code, string input)`. Each implementation handles compilation, execution, and output capture for a specific language.

csharpExecutor : CodeExecutor

Uses `Microsoft.CodeAnalysis.CSharp` (Roslyn) to parse and compile C# code into an in-memory assembly. Wraps the user code in a synthetic entry point, redirects `Console.In/Console.Out` to capture input/output, executes via reflection, and returns the captured output.

JavaExcuteor : CodeExecutor

A stub implementation that returns "Java Output". Provides a placeholder for future Java support.

CodeRunner (Service)

Orchestrates the full test execution pipeline. Accepts user code, language selection, and a list of `TestCase` objects. For each test case, it invokes the appropriate `CodeExecutor`, compares actual output to expected output (case-insensitive, whitespace-normalized), and returns a `CodeRunnerTestResult` list with pass/fail status.

ConnactionCmd

A utility that wraps `System.Diagnostics.Process` for executing external programs with redirected standard input/output. Supports `call(fileName, args)` for parameterless execution and `callWithInput(fileName, args, input)` for providing stdin. Provides friendly error messages when compiler executables are not found.

7.3 Code Compilation & Execution Flow (C#)

1. User writes C# code in SubmittForm and clicks Run
2. SubmittForm fires TestResultsReady event with code and language
3. MainForm calls CodeRunner.RunTestCases(code, language, testCases)
4. For each TestCase:
 - a. CodeRunner calls csharpExecutor.ExecuteCode(code, testCase.Input)
 - b. csharpExecutor parses code with CSharpParseOptions
 - c. Compiles to CSharpCompilation object (in-memory)
 - d. Emits assembly to MemoryStream
 - e. Creates AppDomain or uses reflection to invoke the assembly
 - f. Redirects Console.Out to capture output
 - g. Redirects Console.In to provide test case input
 - h. Returns captured output string
5. CodeRunner normalizes and compares output (case-insensitive)
6. CodeRunnerTestResultList is returned with AllPassed flag
7. Based on AllPassed: navigate to ExpectedForm or Faild

7.4 Test Result Model

```
CodeRunnerTestResult {  
    Passed: bool,  
    TestCase: TestCase,  
    ActualOutput: string,  
    ExpectedOutput: string  
}  
  
CodeRunnerTestResultList {  
    Results: List<CodeRunnerTestResult>,  
    AllPassed: bool  
}
```

7.5 Output Comparison Logic

The CodeRunner applies the following normalization before comparing actual vs. expected output:

- Trims leading and trailing whitespace from both strings.
- Replaces all consecutive whitespace sequences (spaces, tabs, newlines) with a single space.
- Performs a case-insensitive ordinal comparison.
- A test is considered PASSED if the normalized outputs match exactly.

8. Data Flow & Navigation

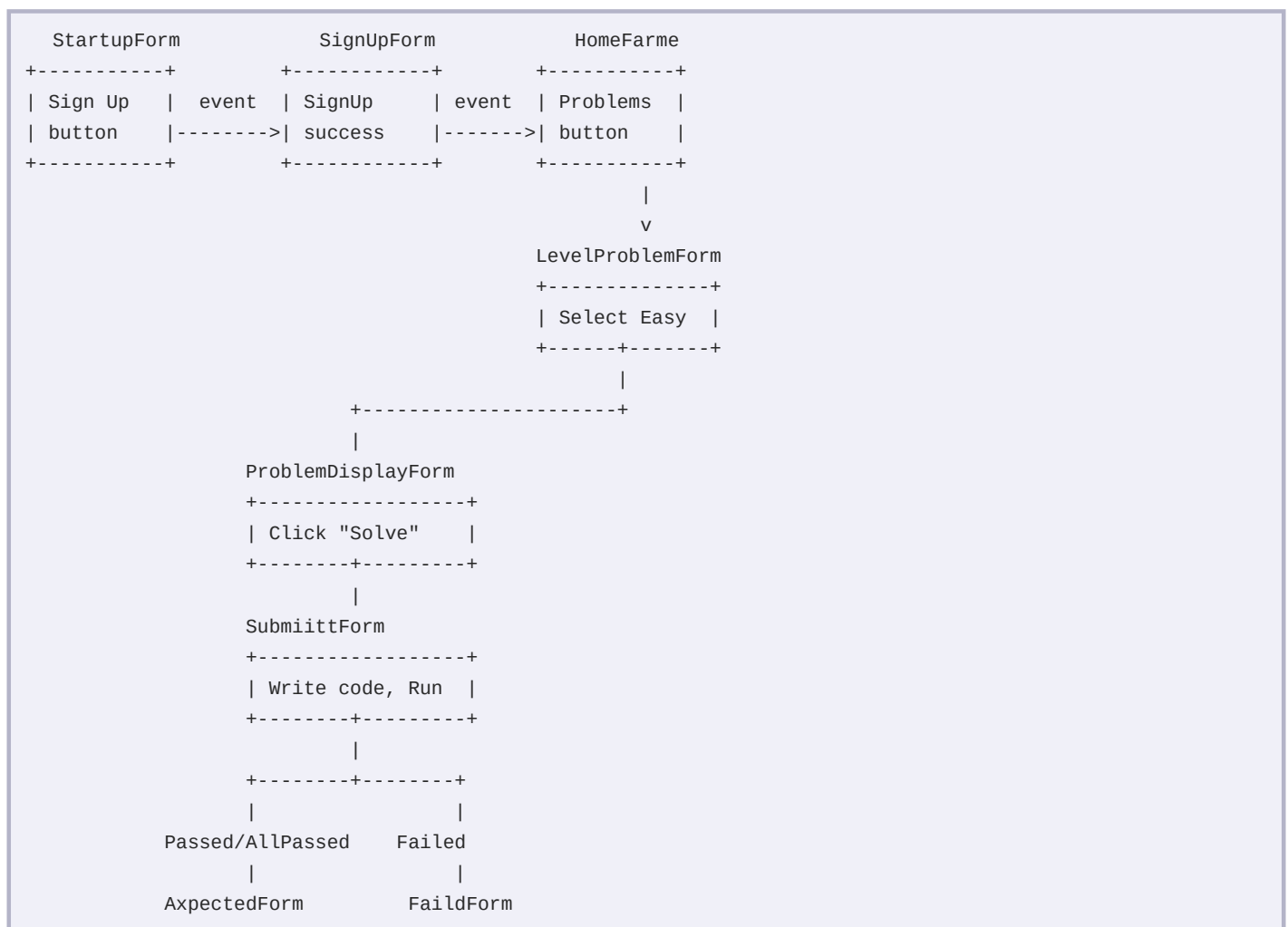
8.1 Event-Driven Navigation

MiniCamp Puzzle uses a custom page-stack navigation model managed by MainForm. The pattern is:

Navigation Stack (backStack)

A List<UserControl> that tracks navigation history. When navigating forward, the current page is pushed onto the stack. GoBack() pops the last page and restores it. This enables standard back-button functionality throughout the application.

8.2 Key Event Flow: Sign Up to Problem Solving



8.3 Session State

The **CurrentUser** static class acts as the session store. After successful login or sign-up, user details (Username, Country, Gender, score) are populated. This data is accessed globally by any component without needing to pass it through constructors. The session is cleared implicitly when the application exits.

```
CurrentUser (static) {  
    Username: string,  
    Country: string,  
    Gender: string,  
    score: int  
}
```

8.4 Problem Loading Flow

Problems are loaded from the embedded JSON file (ProblemJs.json) on demand. The flow is:

- ProblemLoadReadJs reads the JSON file using JavaScriptSerializer.
- Deserializes into List<Problem> object graph.
- LevelProblemForm filters problems by selected difficulty level.
- ProblemDisplayForm retrieves the full problem by title via getProblemByName().
- SubmittForm passes the problem title to CodeRunner, which retrieves test cases from the same model.

9. Security Considerations

9.1 Current Security Posture

The application has the following security characteristics:

Authentication

Username/password authentication against a local SQLite database. No password hashing is implemented; passwords are stored as plaintext. This is acceptable for a local educational tool but constitutes a security risk in any shared or production environment.

Code Execution Isolation

User-submitted C# code is compiled and executed in-memory using the Roslyn compiler API. While this avoids writing executables to disk, the code runs within the same process and AppDomain as the application itself. A malicious user could potentially access the file system, database, or network through submitted code.

SQL Injection

The application uses parameterized SQL queries for all database operations (see `UserManagement.cs`), which prevents SQL injection attacks.

9.2 Recommended Security Enhancements

- Implement password hashing using bcrypt or PBKDF2 before storage.
- Execute user-submitted code in a sandboxed AppDomain with restricted permissions (`SecurityPermissionFlag.Execution`).
- Restrict the types and members accessible through Roslyn compilation using `SystemKey` and source reference filtering.
- Apply input validation limits (max code length, max execution time) to prevent resource exhaustion.
- Consider code execution in a separate process with reduced privileges.

10. Deployment

10.1 Build Process

The project is built using Visual Studio 2022 or MSBuild command line. The solution file (AdvancedProgramming.sln) targets .NET Framework 4.7.2 and produces a WinExe output (AdvancedProgramming.exe). NuGet packages are restored during build from the packages/ directory.

10.2 Prerequisites

- Windows 7 or later operating system.
- .NET Framework 4.7.2 or later runtime.
- No additional runtime dependencies (SQLite and Roslyn are bundled via NuGet).

10.3 Deployment Methods

Visual Studio Publish

ClickOnce publish configured in the project file with PublishUrl = publish\. Produces a self-contained deployment package.

XCOPY Deployment

Build output from bin/Release/ can be copied directly to the target machine. Requires that all DLL dependencies (SQLite, Roslyn assemblies) are included.

Standalone Executable

With additional tooling (e.g., ILMerge, Costura.Fody), dependencies can be embedded into a single executable for simplified distribution.

10.4 Project File Structure (Output)

AdvancedProgramming.exe (main executable)
System.Data.SQLite.dll (SQLite database provider)
Microsoft.CodeAnalysis.dll (Roslyn compiler platform)
Microsoft.CodeAnalysis.CSharp.dll
Microsoft.CodeAnalysis.CSharp.Scripting.dll
Microsoft.CodeAnalysis.Scripting.dll
System.Collections.Immutable.dll
System.Reflection.Metadata.dll
System.Runtime.CompilerServices.Unsafe.dll
System.Threading.Tasks.Extensions.dll
System.Memory.dll
System Buffers.dll
System.Numerics.Vectors.dll
System.Text.Encoding.CodePages.dll
Database/app.db (SQLite database, created on first run)
Image/ (image assets)
ProblemClasses/ProblemJs.json (problem definitions)

11. Future Enhancements

The following enhancements are identified for future versions of MiniCamp Puzzle:

Additional Programming Languages

Extend the JavaExcuteor stub to support actual Java compilation (via command-line javac), and add support for Python, C++, or JavaScript execution.

Unit Test Framework

Add a dedicated test project (JUnit or xUnit) with comprehensive unit tests for CodeRunner, UserManagement, and Theme application logic.

Extended Problem Library

Add more problems across all difficulty levels, supporting additional categories such as dynamic programming, graphs, trees, and string manipulation. Support user-created problems.

Score & Progress Tracking

Implement persistent scoring with leaderboards, track solved/attempted problems, and provide progress analytics.

Rich Code Editor

Replace the plain TextBox code editor with a syntax-highlighting editor (e.g., ScintillaNET, FastColoredTextBox, or a WebView2-based Monaco editor).

Online Multiplayer Mode

Add networking capabilities for real-time competitive coding challenges between users.

Code Execution Sandboxing

Implement proper sandboxed execution for user-submitted code (separate AppDomain or process with restricted permissions) to improve security.

Cloud Save & Sync

Integrate cloud storage (e.g., Firebase, Azure) for cross-device progress sync and online profile management.

Accessibility Improvements

Add high-contrast theme, keyboard navigation, screen reader support, and scalable UI.